SEAH ZHANG JIAN

A24CS0297

Lab 1: Python Fundamentals and a Reproducible Project

Public-health genomics laboratory: build a repository that runs the same way on every laptop, then turn amplicon base counts into a GC report

Item	Detail
Mode	Individual
Weight	Formative (no marks in the current schedule)
Teaching week	Week 1 (12 - 18 Oct 2026)
CLOs	CLO1
Competencies	C01, C09, C10, C11, C18
AI level	Level 1: AI as tutor (PROPOSED policy). You may ask AI to explain concepts, give hints, explain an error message, or review code you wrote. You may NOT paste in the task and submit AI-written code or text as your answer.
AI log	Submit AI_USE_LOG.md (prompt log), or a signed nil return if you used no AI.

Overview

A public-health genomics laboratory wants every analyst's laptop to give the same numbers. Before any real analysis, you set up a small repository with a fixed layout, a pinned environment and a program that reports exactly what Python and which package versions it is running. Then you write three small, tested functions that turn the base counts of an amplicon (how many A, C, G and T it contains) into a length and a GC percentage.

This lab is formative: it carries no marks in the course schedule, but it is checked by the same automatic marker as the later labs, so you see exactly what a marked lab will look like. All data in this course are synthetic; no real patient or laboratory records are used.

Work in the starter repository. Every function has a name, a signature and a docstring. Replace the line `raise NotImplementedError` with your own code. Do not change the function names or signatures.

What you will practise

- Create a project with the standard layout (src, tests, data, results) and a virtual environment with pinned requirements.
- Report the Python version and package versions with importlib.metadata, and write a README with the exact run commands.
- Use variables, int and float, the operators / // and %, f-strings, and guard against division by zero.
- Validate input text before converting it and raise ValueError with a clear message.
- Run pytest and ruff, and add tests of your own for one boundary case and one invalid input.

Files and resources

- Starter repository: src/env_check.py and src/basics.py (stubs), tests/test_visible.py, data/qc_records.csv, pytest.ini, requirements.txt, .gitignore, AI_USE_LOG.md.
- Run the tests with: python -m pytest. Check style with: ruff check src.
- Week 1 lecture notes: environment, repository layout, README, pathlib, importlib.metadata; Python fundamentals.

Instructions

- Unzip the starter, open the folder in VS Code and create a virtual environment inside it (python -m venv .venv). Activate it and run pip install -r requirements.txt.
- Complete the three tasks T1 to T3 in order. Commit after each task with a clear message (git init first; .venv and results/ must stay out of Git).
- Add at least three tests of your own in tests/test_lab01_own.py (see T4).
- Complete AI_USE_LOG.md. This lab is AI Level 1 (tutor): AI may explain concepts, give hints and explain error messages, but must not write your answers.

Tasks

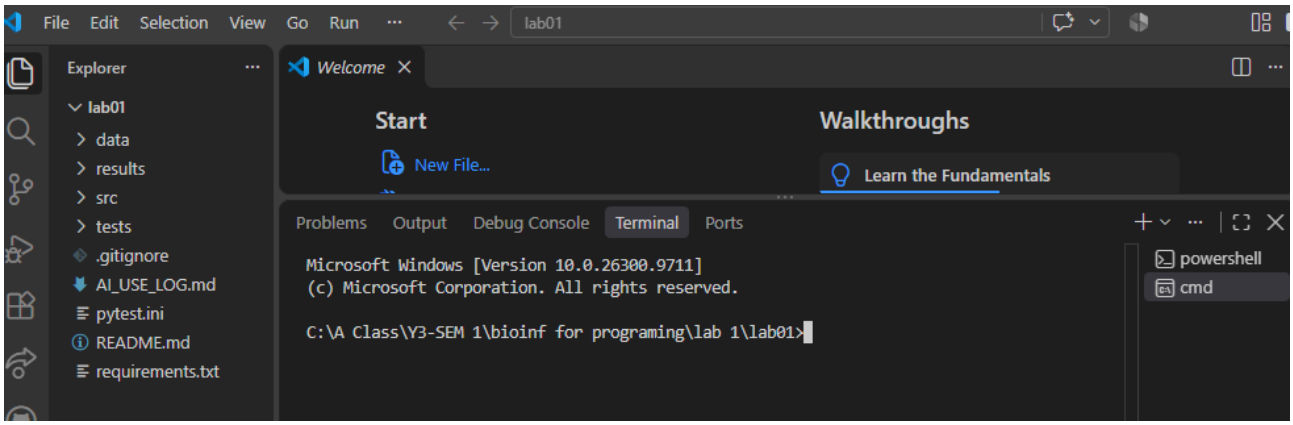
T1. Repository layout, environment record and README (3 marks)

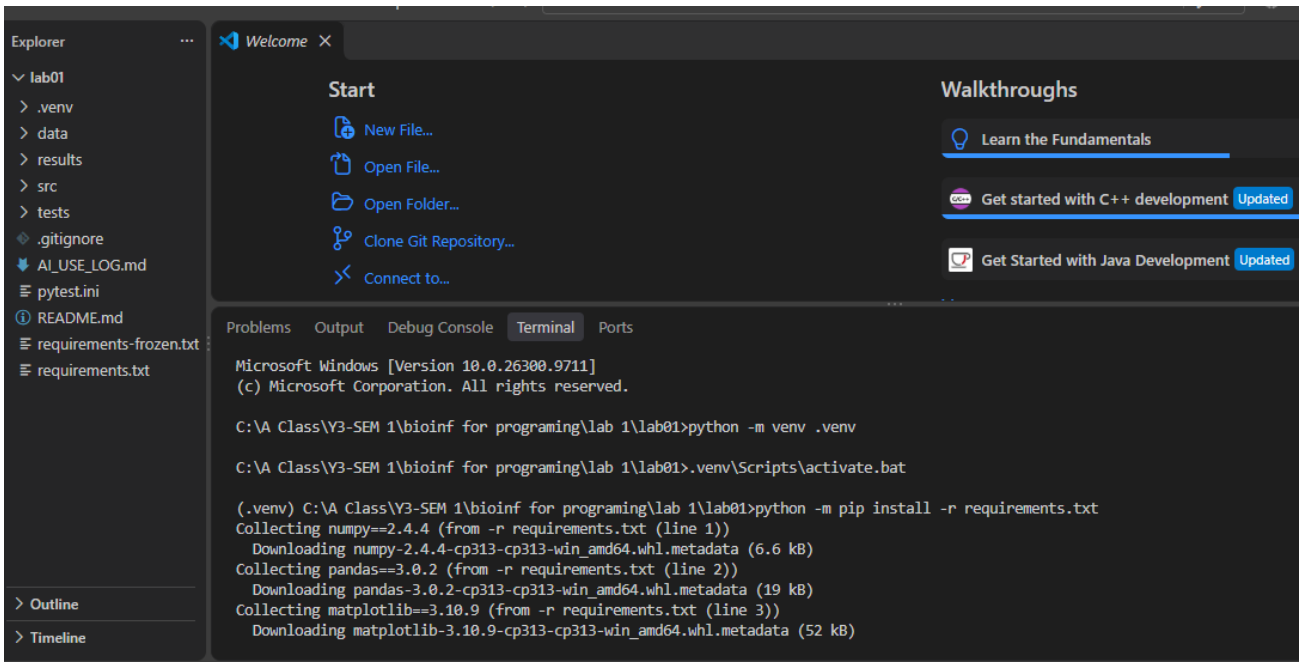
Your repository must contain src/, tests/, data/ and results/ folders, pytest.ini, .gitignore and a README.md that tells a colleague exactly what to type. Record the environment you actually ran: (1) pip freeze > requirements-frozen.txt and (2) python src/env_check.py > results/environment_report.txt (after T2 works). The README must contain the commands to create the environment, install requirements.txt, run `python -m pytest`, run `ruff check src` and run the environment report, plus one line per folder and a short note on what is not committed and why. Use the README template and replace every line in angle brackets.

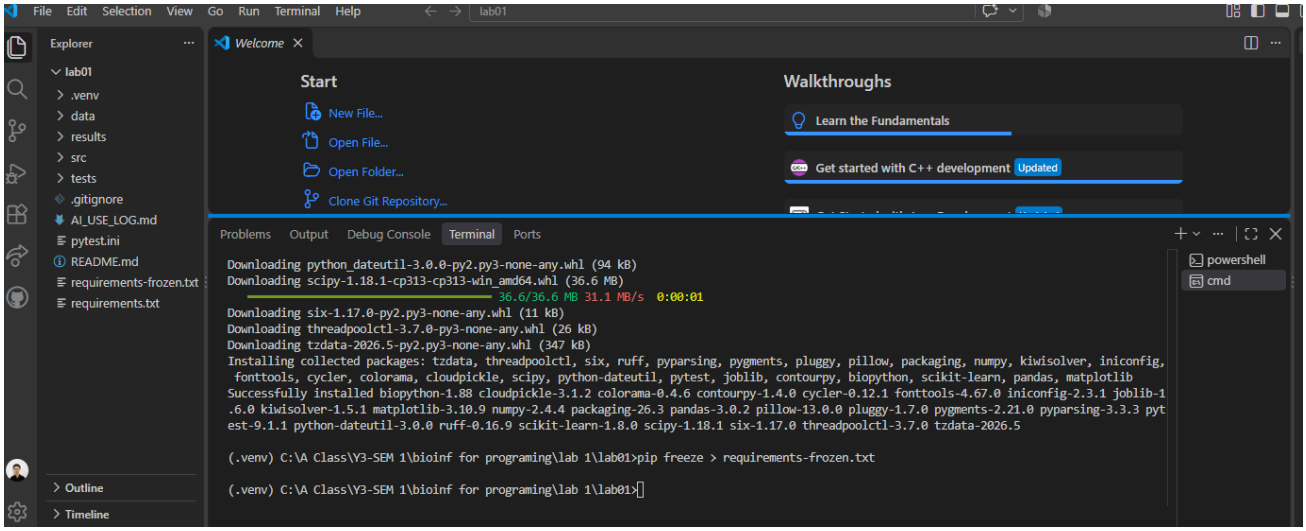
1. Create .venv, activate it, install requirements.txt, run pip freeze > requirements-frozen.txt.
2. Check that data/qc_records.csv is present and never edit it (raw data are read-only).
3. Write the README: setup, run, layout, what is not committed.

Hint: On Windows PowerShell the activate script may be blocked; use activate.bat in cmd. Say in the README which one you used.

Answer:







Task 1: Virtual environment (.venv) creation, package installation from requirements.txt, and snapshot recording via pip freeze into requirements-frozen.txt.

T2. env_check.py: report the environment (4 marks)

package_version(name) returns the installed version string of a package, or None when it is not installed (catch importlib.metadata.PackageNotFoundError; never a bare except). check_environment(packages=PACKAGES) returns a dict with three keys: "python" (platform.python_version()), "platform" (platform.system()) and "packages" (a dict mapping each requested package name to its version or None). Running `python src/env_check.py` prints the report and exits with code 0.

Hint: The pip name is biopython, not bio-python. A wrong name gives None, not a crash.

ANSWER

Take a screenshot of the terminal displaying the table from type results\environment_report.txt

T3. basics.py: base counts to a GC report (6 marks)

gc_percent(a, c, g, t) returns the GC percentage (0 to 100) as a float from the four base counts. Raise ValueError if any count is negative or the total is zero.

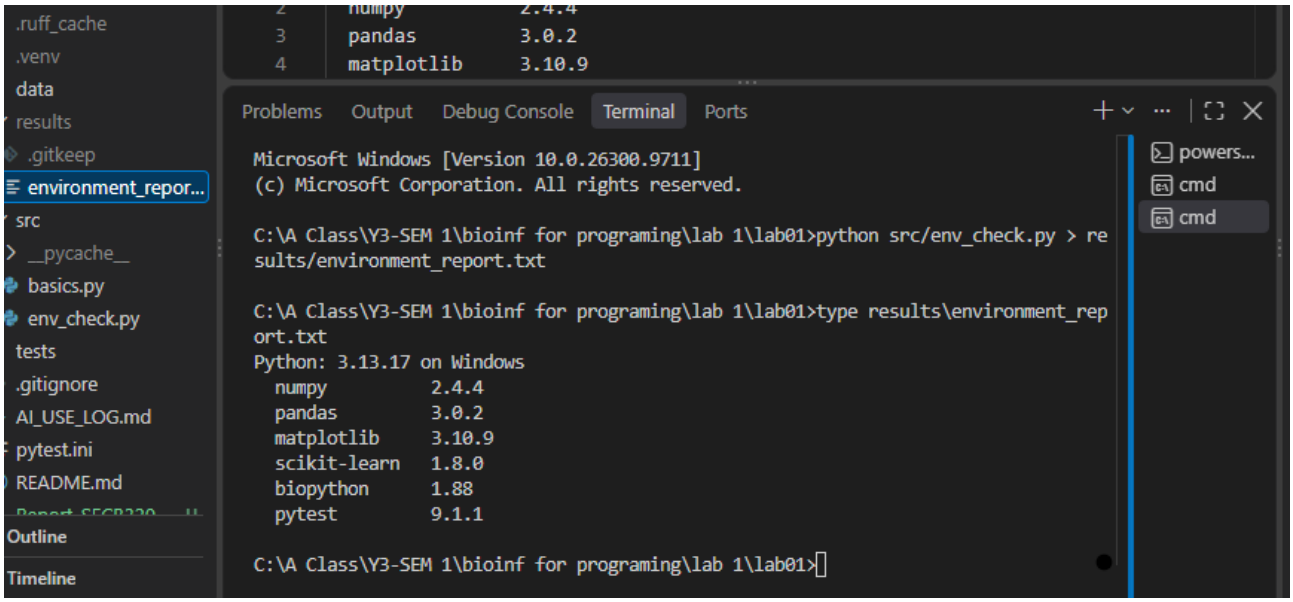
format_report(sample, length, gc) returns a one-line report with units and one decimal place, for example "Amplicon_01: 60 bases, GC 36.7%". It returns the text; it does not print.

parse_counts(text) turns text such as "18 12 10 20" or "18,12,10,20" into a tuple of four ints (A, C, G, T). Raise ValueError with a clear message if there are not exactly four whole numbers or any is negative.

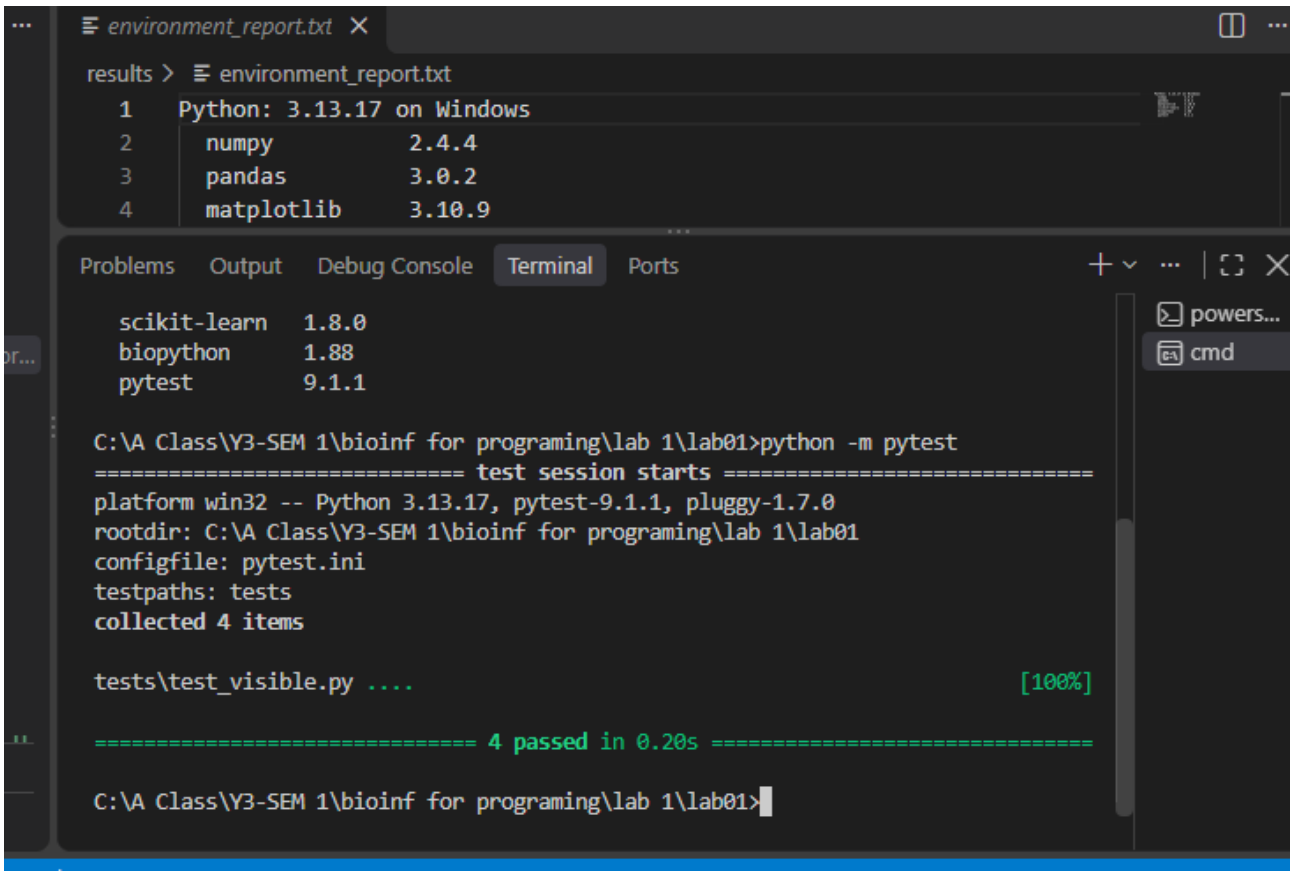
```
>>> gc_percent(18, 12, 10, 20)
36.666666666666664
>>> format_report("Amplicon_01", 60, 36.6667)
'Amplicon_01: 60 bases, GC 36.7%'
>>> parse_counts("18, 12, 10, 20")
(18, 12, 10, 20)
```

Hint: Use / (true division), not //. Test the total for zero before dividing. text.replace(',', ' ').split() handles both separators.

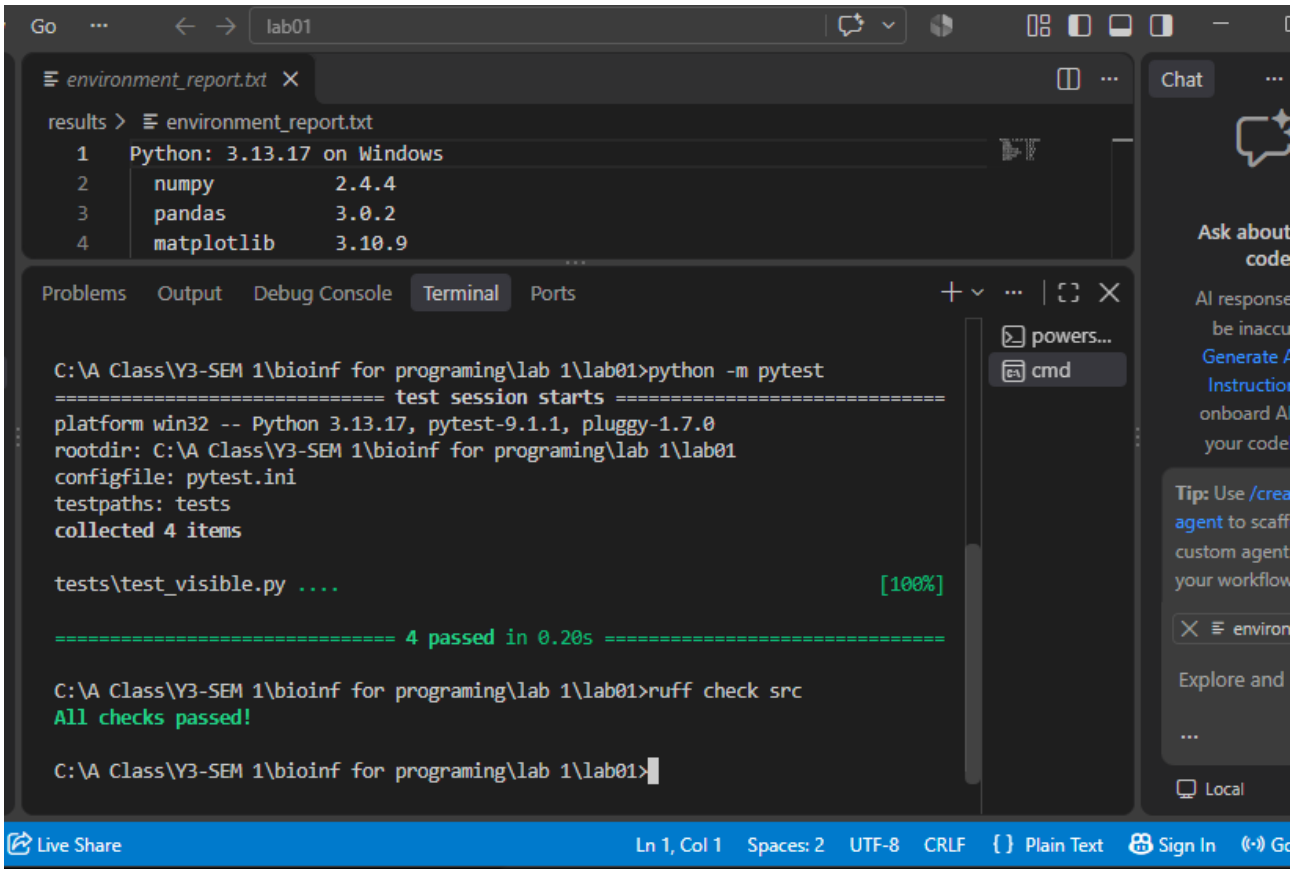
ANSWER



Take a screenshot of the terminal displaying the table from type results\environment_report.txt.



Capture the green output showing all tests passed

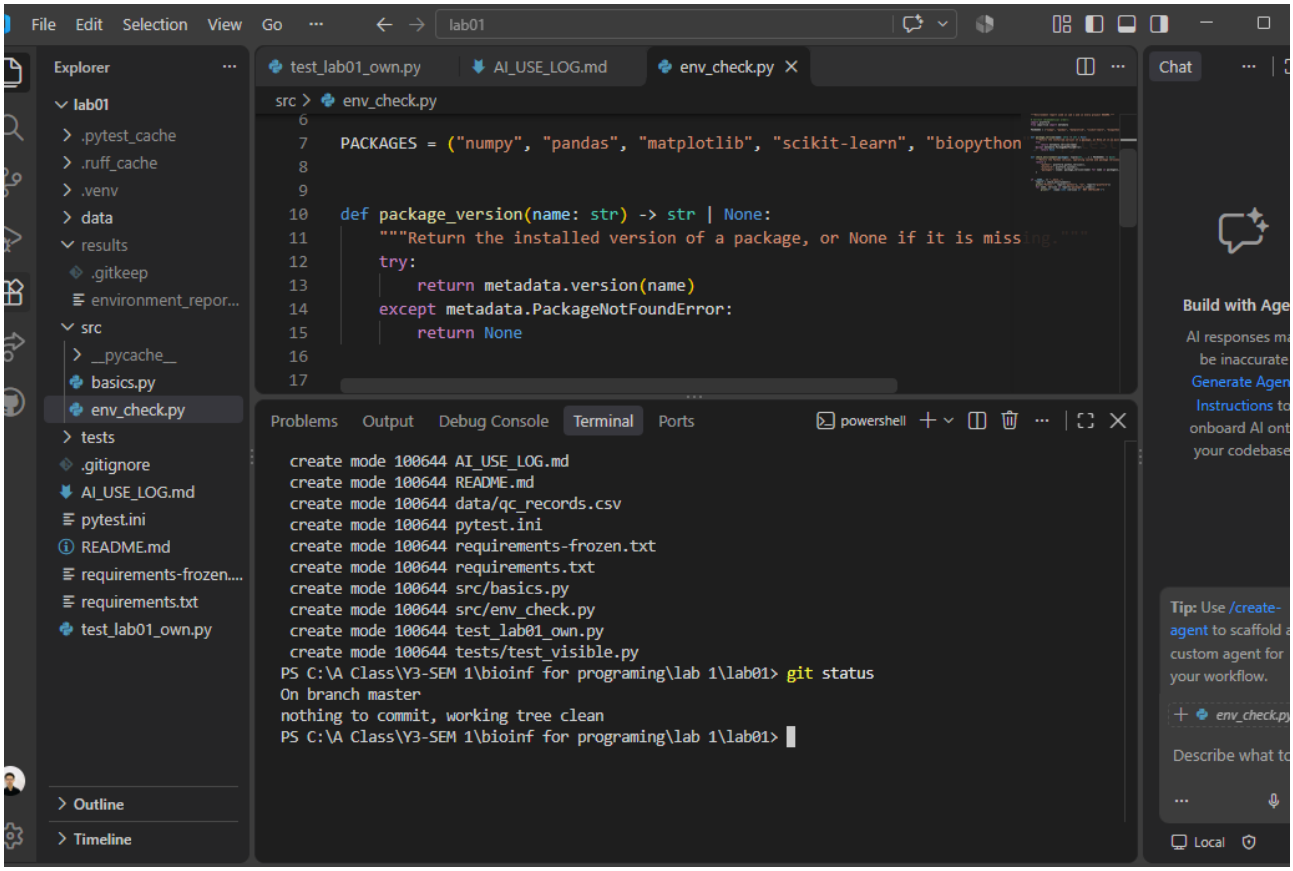


Showing ruff check src with the green message **All checks passed.**

T4. Your own tests, clean code, AI log (4 marks)

Create tests/test_lab01_own.py with at least three tests of your own: one boundary case (for example all counts in G and C), one zero-total case and one invalid-input case. Make `python -m pytest` pass, make `ruff check src` report no issues and complete AI_USE_LOG.md.

ANSWER



Capture the terminal displaying nothing to commit, working tree clean.

Repository layout

Path	What goes there	Committed to Git?
src/	Your modules: env_check.py and basics.py	Yes
tests/	test_visible.py (given) and test_lab01_own.py (yours)	Yes
data/	Read-only input such as qc_records.csv	Small synthetic files only
results/	Files your code generates (environment_report.txt)	No (in .gitignore)
pytest.ini, requirements.txt, .gitignore, README.md	Project settings and instructions	Yes
.venv/	Your virtual environment	Never

What to submit

- Push your repository to GitHub (or submit the zip) before the lab deadline set in the course schedule. Submit a zip if you want results/ marked, because Git ignores that folder on purpose.
- Include src/env_check.py, src/basics.py, tests/test_lab01_own.py, README.md, requirements-frozen.txt, results/environment_report.txt and AI_USE_LOG.md. Do not include .venv.

How it is marked

#	Criterion	Marks	Marked by
R1	env_check.py correct: package_version, check_environment keys, missing package gives None	3	Auto
R2	gc_percent correct: true division, boundaries, ValueError on zero total or negative count	3	Auto
R3	format_report and parse_counts correct, including separators and invalid input	3	Auto
R4	Repository layout and environment record present (folders, pytest.ini, .gitignore, requirements-frozen.txt, environment_report.txt)	2	Auto
R5	README contains the exact commands (venv, pip install, pytest, ruff, environment report)	1	Auto
R6	Own tests: tests/test_lab01_own.py has at least three tests and passes	1	Auto
R7	python src/env_check.py runs from the repository root and exits with code 0	1	Auto
R8	ruff reports no issues on src/	2	Auto
R9	AI use log submitted (prompt log or signed nil return) and valid	1	Auto
R10	Individual Q&A: explains one guard clause, one test and why the environment and paths are set up this way	3	Lecturer
	Total	20	

Automated checks use hidden tests as well as the visible ones, so passing the visible tests alone does not guarantee full marks. You must be able to explain your own code in the individual Q&A.